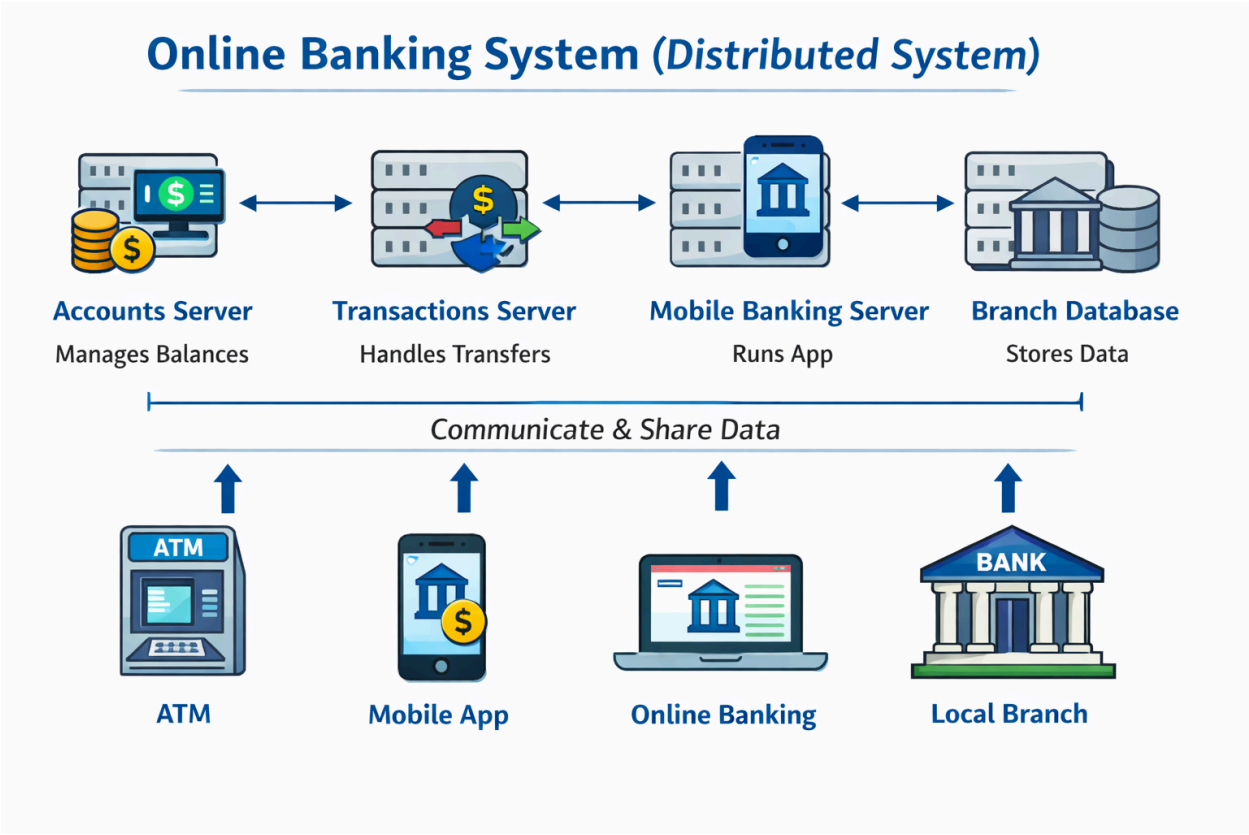
Distributed Computing is a computing model where multiple independent computers (nodes) work together over a network to solve a problem or provide a service, appearing to users as a single system.

Examples of Distributed Computing

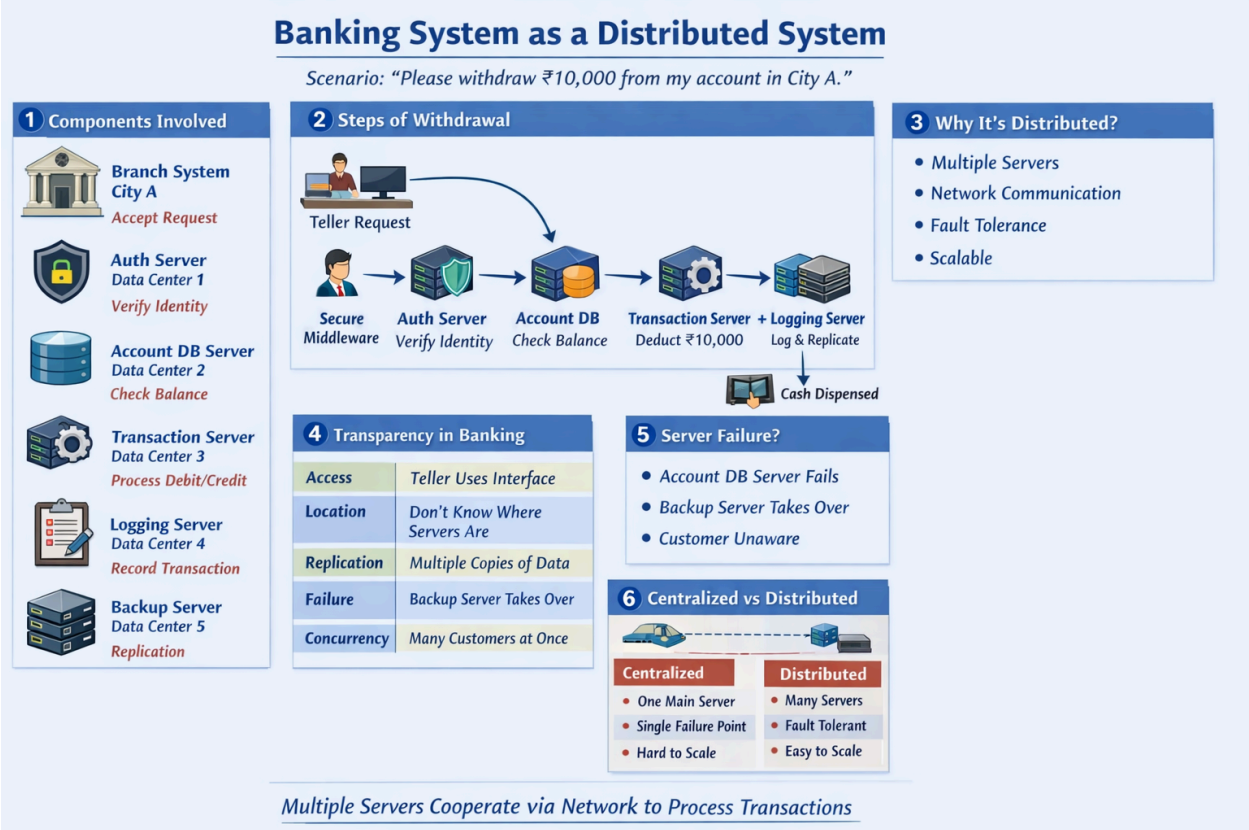
- Google Search
- Cloud computing (AWS, Azure)
- Distributed file systems (HDFS)
- Online multiplayer games

A distributed system is a collection of independent computers that appears to its users as a single coherent system.

https://www.youtube.com/watch?v=DO5F7b7TRC4
https://youtube.com/shorts/HEay_AK_0Ck?si=xVlBexn3vAc_07gt


Online Banking System (Distributed System)
Accounts Server
Manages Balances
Transactions Server
Handles Transfers
Mobile Banking Server
Runs App
Branch Database
Stores Data
Communicate & Share Data
ATM
BANK
ATM
Mobile App
Online Banking
Local Branch

Example:



Issues and Goals of Distributed System

Goals of a Distributed System

A distributed system is designed to achieve the following objectives:

1. Resource Sharing

The system allows sharing of hardware, software, and data among multiple users and computers.
Example: Multiple computers accessing the same centralized database.

2. Transparency

The system should appear as a single unified system to the user, even though it consists of multiple machines.
Types include:

- Location transparency
- Replication transparency
- Failure transparency
- Concurrency transparency

3. Scalability

The system should support growth in terms of users, resources, and geographical expansion without performance degradation.

4. Reliability and Fault Tolerance

The system should continue functioning even if some components fail. Backup and replication techniques are used to achieve this.

5. Performance

Tasks can be divided among multiple machines to improve response time and throughput.

6. Openness

The system should support standard protocols and allow interoperability between different platforms and technologies.

Issues (Challenges) in Distributed Systems

While designing distributed systems, several challenges arise:

1. Heterogeneity

Different hardware, operating systems, networks, and programming languages must work together.

2. Security

Ensuring authentication, authorization, confidentiality, and data integrity across the network.

3. Failure Handling

Failures such as server crash, network failure, or communication breakdown must be detected and handled properly.

4. Concurrency

Multiple users accessing shared resources simultaneously may cause inconsistencies or race conditions.

5. Scalability Issues

As the system grows, maintaining performance and avoiding bottlenecks becomes challenging.

6. Data Consistency

Maintaining uniform data across multiple replicated nodes is difficult.

7. Clock Synchronization

Each machine has its own clock, making event ordering and coordination complex.